

缺少声明可能导致程序行为异常

值得注意的是, 为了使 `char*` 版本的 `debug_rep` 正确工作, 在定义此版本时, `debug_rep(const string&)` 的声明必须在作用域中。否则, 就可能调用错误的 `debug_rep` 版本:

699

```
template <typename T> string debug_rep(const T &t);
template <typename T> string debug_rep(T *p);
// 为了使 debug_rep(char*) 的定义正确工作, 下面的声明必须在作用域中
string debug_rep(const string &);
string debug_rep(char *p)
{
    // 如果接受一个 const string& 的版本的声明不在作用域中,
    // 返回语句将调用 debug_rep(const T&) 的 T 实例化为 string 的版本
    return debug_rep(string(p));
}
```

通常, 如果使用了一个忘记声明的函数, 代码将编译失败。但对于重载函数模板的函数而言, 则不是这样。如果编译器可以从模板实例化出与调用匹配的版本, 则缺少的声明就不重要了。在本例中, 如果忘记了声明接受 `string` 参数的 `debug_rep` 版本, 编译器会默认地实例化接受 `const T&` 的模板版本。



在定义任何函数之前, 记得声明所有重载的函数版本。这样就不必担心编译器由于未遇到你希望调用的函数而实例化一个并非你所需的版本。

16.3 节练习

练习 16.48: 编写你自己版本的 `debug_rep` 函数。

练习 16.49: 解释下面每个调用会发生什么:

```
template <typename T> void f(T);
template <typename T> void f(const T*);
template <typename T> void g(T);
template <typename T> void g(T*);
int i = 42, *p = &i;
const int ci = 0, *p2 = &ci;
g(42); g(p); g(ci); g(p2);
f(42); f(p); f(ci); f(p2);
```

练习 16.50: 定义上一个练习中的函数, 令它们打印一条身份信息。运行该练习中的代码。如果函数调用的行为与你预期不符, 确定你理解了原因。



16.4 可变参数模板

C++
11

一个可变参数模板 (variadic template) 就是一个接受可变数目参数的模板函数或模板类。可变数目的参数被称为参数包 (parameter packet)。存在两种参数包: 模板参数包 (template parameter packet), 表示零个或多个模板参数; 函数参数包 (function parameter packet), 表示零个或多个函数参数。

700

我们用一个省略号来指出一个模板参数或函数参数表示一个包。在一个模板参数列表

中, `class...` 或 `typename...` 指出接下来的参数表示零个或多个类型的列表; 一个类型名后面跟一个省略号表示零个或多个给定类型的非类型参数的列表。在函数参数列表中, 如果一个参数的类型是一个模板参数包, 则此参数也是一个函数参数包。例如:

```
// Args 是一个模板参数包; rest 是一个函数参数包
// Args 表示零个或多个模板类型参数
// rest 表示零个或多个函数参数
template <typename T, typename... Args>
void foo(const T &t, const Args& ... rest);
```

声明了 `foo` 是一个可变参数函数模板, 它有一个名为 `T` 的类型参数, 和一个名为 `Args` 的模板参数包。这个包表示零个或多个额外的类型参数。`foo` 的函数参数列表包含一个 `const &` 类型的参数, 指向 `T` 的类型, 还包含一个名为 `rest` 的函数参数包, 此包表示零个或多个函数参数。

与往常一样, 编译器从函数的实参推断模板参数类型。对于一个可变参数模板, 编译器还会推断包中参数的数目。例如, 给定下面的调用:

```
int i = 0; double d = 3.14; string s = "how now brown cow";
foo(i, s, 42, d);      // 包中有三个参数
foo(s, 42, "hi");      // 包中有两个参数
foo(d, s);             // 包中有一个参数
foo("hi");             // 空包
```

编译器会为 `foo` 实例化出四个不同的版本:

```
void foo(const int&, const string&, const int&, const double&);
void foo(const string&, const int&, const char[3]&);
void foo(const double&, const string&);
void foo(const char[3]&);
```

在每个实例中, `T` 的类型都是从第一个实参的类型推断出来的。剩下的实参 (如果有的话) 提供函数额外实参的数目和类型。

sizeof...运算符

当我们需要知道包中有多少元素时, 可以使用 `sizeof...` 运算符。类似 `sizeof` (参见 4.9 节, 第 139 页), `sizeof...` 也返回一个常量表达式 (参见 2.4.4 节, 第 58 页), 而且不会对其实参求值:

```
template<typename ... Args> void g(Args ... args) {
    cout << sizeof...(Args) << endl; // 类型参数的数目
    cout << sizeof...(args) << endl; // 函数参数的数目
}
```

C++
11

16.4 节练习

701

练习 16.51: 调用本节中的每个 `foo`, 确定 `sizeof...(Args)` 和 `sizeof...(rest)` 分别返回什么。

练习 16.52: 编写一个程序验证上一题的答案。

16.4.1 编写可变参数函数模板

如 6.2.6 节（第 198 页）所述，我们可以使用一个 `initializer_list` 来定义一个可接受可变数目实参的函数。但是，所有实参必须具有相同的类型（或它们的类型可以转换为同一个公共类型）。当我们既不知道想要处理的实参的数目也不知道它们的类型时，可变参数函数是很有用的。作为一个例子，我们将定义一个函数，它类似较早的 `error_msg` 函数，差别仅在于新函数实参的类型也是可变的。我们首先定义一个名为 `print` 的函数，它在一个给定流上打印给定实参列表的内容。

可变参数函数通常是递归的（参见 6.3.2 节，第 204 页）。第一步调用处理包中的第一个实参，然后用剩余实参调用自身。我们的 `print` 函数也是这样的模式，每次递归调用将第二个实参打印到第一个实参表示的流中。为了终止递归，我们还需要定义一个非可变参数的 `print` 函数，它接受一个流和一个对象：

```
// 用来终止递归并打印最后一个元素的函数
// 此函数必须在可变参数版本的 print 定义之前声明
template<typename T>
ostream &print(ostream &os, const T &t)
{
    return os << t; // 包中最后一个元素之后不打印分隔符
}
// 包中除了最后一个元素之外的其他元素都会调用这个版本的 print
template <typename T, typename... Args>
ostream &print(ostream &os, const T &t, const Args&... rest)
{
    os << t << ", "; // 打印第一个实参
    return print(os, rest...); // 递归调用，打印其他实参
}
```

第一个版本的 `print` 负责终止递归并打印初始调用中的最后一个实参。第二个版本的 `print` 是可变参数版本，它打印绑定到 `t` 的实参，并调用自身来打印函数参数包中的剩余值。

这段程序的关键部分是可变参数函数中对 `print` 的调用：

```
return print(os, rest...); //递归调用，打印其他实参
```

我们的可变参数版本的 `print` 函数接受三个参数：一个 `ostream&`，一个 `const T&` 和一个参数包。而此调用只传递了两个实参。其结果是 `rest` 中的第一个实参被绑定到 `t`，剩余实参形成下一个 `print` 调用的参数包。因此，在每个调用中，包中的第一个实参被移除，成为绑定到 `t` 的实参。即，给定：

```
print(cout, i, s, 42); // 包中有两个参数
```

递归会执行如下：

调用	t	rest...
print(cout, i, s, 42)	i	s, 42
print(cout, s, 42)	s	42
print(cout, 42)	调用非可变参数版本的 print	

前两个调用只能与可变参数版本的 `print` 匹配，非可变参数版本是不可行的，因为这两个调用分别传递四个和三个实参，而非可变参数 `print` 只接受两个实参。

对于最后一次递归调用 `print(cout, 42)`，两个 `print` 版本都是可行的。这个调用传递两个实参，第一个实参的类型为 `ostream&`。因此，可变参数版本的 `print` 可以实例化为只接受两个参数：一个是 `ostream&` 参数，另一个是 `const T&` 参数。

对于最后一个调用，两个函数提供同样好的匹配。但是，非可变参数模板比可变参数模板更特例化，因此编译器选择非可变参数版本（参见 16.3 节，第 615 页）。



当定义可变参数版本的 `print` 时，非可变参数版本的声明必须在作用域中。否则，可变参数版本会无限递归。

16.4.1 节练习

练习 16.53: 编写你自己版本的 `print` 函数，并打印一个、两个及五个实参来测试它，要打印的每个实参都应有不同的类型。

练习 16.54: 如果我们对一个没有 `<<` 运算符的类型调用 `print`，会发生什么？

练习 16.55: 如果我们的可变参数版本 `print` 的定义之后声明非可变参数版本，解释可变参数的版本会如何执行。

16.4.2 包扩展



对于一个参数包，除了获取其大小外，我们能对它做的唯一的事情就是扩展（`expand`）它。当扩展一个包时，我们还要提供用于每个扩展元素的模式（`pattern`）。扩展一个包就是将它分解为构成的元素，对每个元素应用模式，获得扩展后的列表。我们通过在模式右边放一个省略号（`...`）来触发扩展操作。

703

例如，我们的 `print` 函数包含两个扩展：

```
template <typename T, typename... Args>
ostream &
print(ostream &os, const T &t, const Args&... rest)    // 扩展 Args
{
    os << t << ", ";
    return print(os, rest...);                          // 扩展 rest
}
```

第一个扩展操作扩展模板参数包，为 `print` 生成函数参数列表。第二个扩展操作出现在对 `print` 的调用中。此模式为 `print` 调用生成实参列表。

对 `Args` 的扩展中，编译器将模式 `const Arg&` 应用到模板参数包 `Args` 中的每个元素。因此，此模式的扩展结果是一个逗号分隔的零个或多个类型的列表，每个类型都形如 `const type&`。例如：

```
print(cout, i, s, 42); // 包中有两个参数
```

最后两个实参的类型和模式一起确定了尾置参数的类型。此调用被实例化为：

```
ostream&
print(ostream&, const int&, const string&, const int&);
```

第二个扩展发生在对 `print` 的（递归）调用中。在此情况下，模式是函数参数包的名字（即 `rest`）。此模式扩展出一个由包中元素组成的、逗号分隔的列表。因此，这个调

用等价于：

```
print(os, s, 42);
```

理解包扩展

`print` 中的函数参数包扩展仅仅将包扩展为其构成元素，C++ 语言还允许更复杂的扩展模式。例如，我们可以编写第二个可变参数函数，对其每个实参调用 `debug_rep`（参见 16.3 节，第 615 页），然后调用 `print` 打印结果 `string`：

```
// 在 print 调用中对每个实参调用 debug_rep
template <typename... Args>
ostream &errorMsg(ostream &os, const Args&... rest)
{
    // print(os, debug_rep(a1), debug_rep(a2), ..., debug_rep(an))
    return print(os, debug_rep(rest)...);
}
```

704

这个 `print` 调用使用了模式 `debug_rep(rest)`。此模式表示我们希望对函数参数包 `rest` 中的每个元素调用 `debug_rep`。扩展结果将是一个逗号分隔的 `debug_rep` 调用列表。即，下面调用：

```
errorMsg(cerr, fcnName, code.num(), otherData, "other", item);
```

就好像我们这样编写代码一样

```
print(cerr, debug_rep(fcnName), debug_rep(code.num()),
      debug_rep(otherData), debug_rep("otherData"),
      debug_rep(item));
```

与之相对，下面的模式会编译失败

```
// 将包传递给 debug_rep; print(os, debug_rep(a1, a2, ..., an))
print(os, debug_rep(rest...)); // 错误：此调用无匹配函数
```

这段代码的问题是在我们 `debug_rep` 调用中扩展了 `rest`，它等价于

```
print(cerr, debug_rep(fcnName, code.num(),
                      otherData, "otherData", item));
```

在这个扩展中，我们试图用一个五个实参的列表来调用 `debug_rep`，但并不存在与此调用匹配的 `debug_rep` 版本。`debug_rep` 函数不是可变参数的，而且没有哪个 `debug_rep` 版本接受五个参数。

Note

扩展中的模式会独立地应用于包中的每个元素。

16.4.2 节练习

练习 16.56：编写并测试可变参数版本的 `errorMsg`。

练习 16.57：比较你的可变参数版本的 `errorMsg` 和 6.2.6 节（第 198 页）中的 `error_msg` 函数。两种方法的优点和缺点各是什么？



16.4.3 转发参数包

C++
11

在新标准下，我们可以组合使用可变参数模板与 `forward` 机制来编写函数，实现将

其实参不变地传递给其他函数。作为例子，我们将为 `StrVec` 类（参见 13.5 节，第 465 页）添加一个 `emplace_back` 成员。标准库容器的 `emplace_back` 成员是一个可变参数成员模板（参见 16.1.4 节，第 596 页），它用其实参在容器管理的内存空间中直接构造一个元素。

我们为 `StrVec` 设计的 `emplace_back` 版本也应该是可变参数的，因为 `string` 有多个构造函数，参数各不相同。由于我们希望能使用 `string` 的移动构造函数，因此还需要保持传递给 `emplace_back` 的实参的所有类型信息。 ◀ 705

如我们所见，保持类型信息是一个两阶段的过程。首先，为了保持实参中的类型信息，必须将 `emplace_back` 的函数参数定义为模板类型参数的右值引用（参见 16.2.7 节，第 613 页）：

```
class StrVec {
public:
    template <class... Args> void emplace_back(Args&&...);
    // 其他成员的定义，同 13.5 节（第 465 页）
};
```

模板参数包扩展中的模式是 `&&`，意味着每个函数参数将是一个指向其对应实参的右值引用。

其次，当 `emplace_back` 将这些实参传递给 `construct` 时，我们必须使用 `forward` 来保持实参的原始类型（参见 16.2.7 节，第 614 页）：

```
template <class... Args>
inline
void StrVec::emplace_back(Args&&... args)
{
    chk_n_alloc(); // 如果需要的话重新分配 StrVec 内存空间
    alloc.construct(first_free++, std::forward<Args>(args)...);
}
```

`emplace_back` 的函数体调用了 `chk_n_alloc`（参见 13.5 节，第 465 页）来确保有足够的空间容纳一个新元素，然后调用了 `construct` 在 `first_free` 指向的位置中创建了一个元素。`construct` 调用中的扩展为

```
std::forward<Args>(args)...
```

它既扩展了模板参数包 `Args`，也扩展了函数参数包 `args`。此模式生成如下形式的元素

```
std::forward<Ti>(ti)
```

其中 T_i 表示模板参数包中第 i 个元素的类型， t_i 表示函数参数包中第 i 个元素。例如，假定 `svec` 是一个 `StrVec`，如果我们调用

```
svec.emplace_back(10, 'c'); // 将 cccccccccc 添加为新的尾元素
```

`construct` 调用中的模式会扩展出

```
std::forward<int>(10), std::forward<char>(c)
```

通过在此调用中使用 `forward`，我们保证如果用一个右值调用 `emplace_back`，则 `construct` 也会得到一个右值。例如，在下面的调用中：

```
svec.emplace_back(s1 + s2); // 使用移动构造函数
```

传递给 `emplace_back` 的实参是一个右值，它将以如下形式传递给 `construct`

```
std::forward<string>(string("the end"))
```

`forward<string>`的结果类型是 `string&&`，因此 `construct` 将得到一个右值引用实参。`construct` 会继续将此实参传递给 `string` 的移动构造函数来创建新元素。

706

建议：转发和可变参数模板

可变参数函数通常将它们的参数转发给其他函数。这种函数通常具有与我们的 `emplace_back` 函数一样的形式：

```
// fun 有零个或多个参数，每个参数都是一个模板参数类型的右值引用
template<typename... Args>
void fun(Args&&... args) // 将 Args 扩展为一个右值引用的列表
{
    // work 的实参既扩展 Args 又扩展 args
    work(std::forward<Args>(args)...);
}
```

这里我们希望将 `fun` 的所有实参转发给另一个名为 `work` 的函数，假定由它完成函数的实际工作。类似 `emplace_back` 中对 `construct` 的调用，`work` 调用中的扩展既扩展了模板参数包也扩展了函数参数包。

由于 `fun` 的参数是右值引用，因此我们可以传递给它任意类型的实参；由于我们使用 `std::forward` 传递这些实参，因此它们的所有类型信息在调用 `work` 时都会得到保持。

16.4.3 节练习

练习 16.58: 为你的 `StrVec` 类及你为 16.1.2 节（第 591 页）练习中编写的 `Vec` 类添加 `emplace_back` 函数。

练习 16.59: 假定 `s` 是一个 `string`，解释调用 `svec.emplace_back(s)` 会发生什么。

练习 16.60: 解释 `make_shared`（参见 12.1.1 节，第 401 页）是如何工作的。

练习 16.61: 定义你自己版本的 `make_shared`。



16.5 模板特例化

编写单一模板，使之对任何可能的模板实参都是最适合的，都能实例化，这并不总是能办到。在某些情况下，通用模板的定义对特定类型是不适合的：通用定义可能编译失败或做得不正确。其他时候，我们也可以利用某些特定知识来编写更高效的代码，而不是从通用模板实例化。当我们不能（或不希望）使用模板版本时，可以定义类或函数模板的一个特例化版本。

我们的 `compare` 函数是一个很好的例子，它展示了函数模板的通用定义不适合一个特定类型（即字符指针）的情况。我们希望 `compare` 通过调用 `strcmp` 比较两个字符指针而非比较指针值。实际上，我们已经重载了 `compare` 函数来处理字符串字面常量（参见 16.1.1 节，第 579 页）：